



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
del Estado de Chile

TRABAJO N°1: UN PROBLEMA DE CUADRATURA

Para la asignatura de Computación Paralela y Distribuida

Integrantes:

Luna León Chandia

Mei-Ying Mamani Leon

José Miguel Vargas Moraga

Profesor:

Sebastián Salazar Molina

15 de Junio del 2026

Índice

INTRODUCCIÓN.....	3
DESCRIPCIÓN DETALLADA DEL PROBLEMA.....	4
SOLUCIÓN IMPLEMENTADA.....	6
I. Estructura del Proyecto.....	6
II. Archivos del proyecto.....	7
Tabla 1: Archivos del proyecto.....	7
III. Descripción de las soluciones implementadas.....	8
IV. Diagrama de flujo o pseudocódigo del algoritmo paralelo.....	10
Imagen 1: Diagrama de flujo.....	10
ANÁLISIS DE RENDIMIENTO.....	11
I. Tiempo de ejecución.....	11
Tabla 2: Tiempo de ejecución respecto a la cantidad de hilos.....	11
Tabla 3: Detall.....	11
RESULTADOS DE EJECUCIÓN.....	12
Tabla 4: Promedio de compras por género.....	12
DIFICULTADES ENCONTRADAS Y CÓMO SE RESOLVIERON.....	13
CONCLUSIONES.....	14
ANEXOS.....	15

INTRODUCCIÓN

En el presente informe se desarrolla una solución para el problema propuesto en la asignatura de Computación Paralela y Distribuida, el cual consiste en procesar grandes volúmenes de datos de ventas pertenecientes a una farmacia ficticia llamada Cruz Morada. Estos datos se encuentran almacenados en archivos CSV dentro de un servidor SFTP y contienen información relevante sobre transacciones, productos, montos de compra y códigos únicos de clientes.

El principal desafío del trabajo consiste en diseñar e implementar un programa en C++ que sea capaz de descargar, leer, validar y procesar estos archivos de manera eficiente. Además, el sistema debe complementar la información de cada cliente mediante consultas a una API REST, con el objetivo de obtener su género y, posteriormente, calcular el promedio de compras realizadas por clientes masculinos y femeninos.

Debido al alto volumen de información, que puede alcanzar millones de registros, una solución secuencial resulta poco eficiente y genera tiempos de ejecución demasiado extensos. Por esta razón, se incorpora el uso de paralelismo mediante OpenMP, permitiendo distribuir distintas tareas entre múltiples hilos, como la descarga de archivos, el parseo de datos y las consultas a la API. Asimismo, se implementan mecanismos de optimización, como el uso de caché en memoria para evitar consultas repetidas sobre un mismo cliente.

A lo largo del informe se describe en detalle el problema abordado, la solución implementada, el diseño modular del sistema, las estrategias de paralelización utilizadas y los resultados obtenidos. Finalmente, se analiza el rendimiento de la solución, comparando los tiempos de ejecución con distintas cantidades de hilos, y se exponen las principales dificultades encontradas durante el desarrollo junto con las decisiones tomadas para resolverlas.

DESCRIPCIÓN DETALLADA DEL PROBLEMA

La cadena de farmacias Cruz Morada enfrenta un problema relacionado con el procesamiento eficiente de grandes volúmenes de datos de ventas. Actualmente, sus registros de ventas se almacenan diariamente en archivos CSV dentro de un servidor SFTP, bajo una nomenclatura estandarizada del tipo `reporte_(Año)(Día del año).csv`, por ejemplo `reporte_26128.csv`, que representa el día 128 del año 2026. Cada uno de estos archivos contiene miles de transacciones, por lo que el volumen total de información puede crecer rápidamente y volverse costoso de procesar de manera secuencial.

El objetivo principal del trabajo es desarrollar una solución en C/C++ que permita descargar, leer, procesar y analizar estos archivos de ventas utilizando paralelismo con OpenMP. La solución debe ser capaz de optimizar el tiempo de procesamiento, especialmente considerando que se deben analizar múltiples archivos y muchas transacciones.

Cada archivo CSV contiene información relevante de una compra, como la fecha de la operación, canal de compra, SKU, producto, unidades, porcentaje de descuento, monto aplicado, número de boleta, local y código del cliente. Este último dato, el CODIGO CLIENTE, corresponde a un identificador único en formato UUID y es clave para obtener información adicional del cliente mediante una API externa.

El problema no se limita solamente a leer los archivos CSV. Para poder calcular las métricas solicitadas, el programa debe consultar una API REST usando el UUID del cliente. Esta API devuelve información personal del cliente, incluyendo el campo `gender`, que puede indicar si el cliente corresponde a género FEMENINO o MASCULINO. Con esta información, cada transacción debe asociarse al género correspondiente del cliente.

La métrica principal que se debe calcular es el promedio de compras realizadas por mujeres y por hombres. Para ello, el programa debe tomar el campo `MONTO APLICADO` de cada transacción y agruparlo según el género del cliente obtenido desde la API. Luego, debe calcular el promedio de monto de compra para clientes con género femenino y el promedio para clientes con género masculino. Las transacciones cuyo UUID no exista en la API o cuyo género no esté definido deben ser ignoradas.

Un punto importante del problema es la eficiencia. Debido a que pueden existir muchos archivos CSV y muchos clientes repetidos, la solución debe evitar trabajo innecesario. Por ejemplo, si un mismo UUID aparece en varias transacciones, no debería consultarse la API varias veces para el mismo cliente. Para eso, se espera implementar algún tipo de caché que almacene los resultados ya consultados.

Además, el uso de OpenMP es obligatorio para paralelizar partes del proceso. Se espera que el programa pueda paralelizar tareas como la descarga de archivos, el parseo de los CSV, las consultas a la API y el cálculo de los resultados. Sin embargo, al trabajar con paralelismo, aparece otro problema importante: la sincronización. Como varios hilos pueden acceder al

mismo tiempo a estructuras compartidas, como la caché de UUIDs o los acumuladores de montos, el programa debe proteger esos accesos usando mecanismos como `critical`, `atomic` o `reduction`.

También se exige que la solución sea robusta. Esto significa que debe validar errores de conexión al SFTP, errores en las respuestas de la API, archivos corruptos, formatos incorrectos, datos faltantes o códigos HTTP como 404. Los errores encontrados deben registrarse en un archivo `log.txt`, permitiendo saber qué ocurrió durante la ejecución del programa.

Por último, el programa debe entregar sus resultados de forma clara. Debe imprimir en consola el promedio calculado para género femenino, el promedio para género masculino y el tiempo total de ejecución. Además, estos mismos resultados deben guardarse en un archivo llamado `resultados.txt`. También se debe medir el tiempo total del programa, usando por ejemplo `omp_get_wtime()`, para poder evaluar el rendimiento de la solución paralela.

En base a todo lo explicado anteriormente, se puede decir que el problema consiste en construir un sistema en C/C++ que procese grandes volúmenes de transacciones de farmacia almacenadas en archivos CSV remotos, complemente esos datos mediante una API REST para conocer el género de cada cliente, y calcule de forma eficiente el promedio de monto de compra por género usando paralelismo con OpenMP. El desafío principal no está solo en obtener el resultado correcto, sino también en hacerlo de forma rápida, segura, robusta y bien estructurada.

SOLUCIÓN IMPLEMENTADA

I. Estructura del Proyecto

problema_ComputacionParalela-main/

— Makefile	
— README.md	
— src/	
— main.cpp	# Orquestador y menú principal
— utils/	
— config.h / config.cpp	# Constantes globales
— logger.h / logger.cpp	# Singleton thread-safe para bitácora
— sftp/	
— sftp_client.h / sftp_client.cpp	# Cliente SFTP con libcurl
— csv/	
— transaccion.h	# Struct de datos por fila CSV
— csv_parseo.h / csv_parseo.cpp	# Parser y validador de archivos CSV
— api/	
— api.h / api.cpp	# Autenticación JWT y consulta REST
— paralelismo/	
— sftp_paralelo.h / sftp_paralelo.cpp	# Descarga paralela SFTP
— parseo_paralelo.h / parseo_paralelo.cpp	# Parseo paralelo CSV
— api_paralelo.h / api_paralelo.cpp	# Consultas API paralelas

Dependencias:

- (1) sudo apt update
- (2) sudo apt install build-essential libcurl4-openssl-dev libomp-dev

II. Archivos del proyecto

Módulo	Archivo(s)	Responsabilidad
Configuración	utils/config.h/cpp	Constantes globales: credenciales SFTP, rutas, timeouts, reintentos
Logger	utils/logger.h/cpp	Singleton thread-safe con niveles INFO/WARNING/ERROR/DEBUG y escritura en log.txt
Cliente SFTP	sftp/sftp_client.h/cpp	Conexión, listado y descarga de archivos. Versión thread-safe para OpenMP
Parser CSV	csv/csv_parseo.h/cpp	Lectura, validación y conversión de filas CSV a structs Transacción
Cliente REST	api/api.h/cpp	Autenticación JWT y consulta GET /person/{uuid} para obtener género
Descarga Paralela	paralelismo/sftp_paralelo.h/cpp	Orquesta la descarga masiva con pragma omp parallel for
Parseo Paralelo	paralelismo/parseo_paralelo.h/cpp	Distribuye archivos entre hilos y fusiona resultados con sección crítica
API Paralela	paralelismo/api_paralelo.h/cpp	Consulta paralela con caché, renovación de token y acumuladores por género
Orquestador	main.cpp	Medición de tiempo total y coordinación de fases

Tabla 1: Archivos del proyecto

III. Descripción de las soluciones implementadas

Inicialmente se desarrolló un programa base que realizará las tareas básicas como la descarga, el parseo, la verificación de datos, la consulta a API y el cálculo de métricas de manera completamente secuencial y lineal. De esto, se logró establecer el orden básico de la lógica de trabajo; la ejecución de este código inicial lograba retornar correctamente la cantidad de gasto promedio separada por género, sin embargo, demoraba alrededor de 18 horas en realizar todo el proceso de inicio a final.

Después de el desarrollo del programa secuencial, se desarrolló un programa con paralelismo básico, el programa procesaba secuencialmente los archivos completos de registros de compra, el cual contenían un volumen aproximado de 15 millones de registros. Aunque se intentaba mitigar el impacto de la red utilizando una tabla hash compartida en memoria para evitar consultas duplicadas, la naturaleza asíncrona y desordenada de los hilos de OpenMP provocaba una severa condición de carrera. Cuando múltiples hilos procesan en paralelo transacciones pertenecientes a un mismo cliente que aún no había sido resuelto en internet, los hilos marcaban el registro en estado PENDIENTE, pero entraban en un bucle de espera activa (busy waiting) utilizando la función `usleep`. Esta estrategia genera una alta contención de candados sobre el recurso crítico del mapa global y, al mismo tiempo, gatilló millones de peticiones HTTP redundantes hacia el servidor remoto, multiplicando innecesariamente la carga sobre la infraestructura externa. Además, cada hilo abría y cerraba una conexión de red de forma individual por cada transacción, quedando totalmente expuesto a la latencia de tránsito (RTT).

Para el desarrollo del programa final con implementación de programación paralela, se eligió `libcurl` como única librería de red para cubrir tanto el protocolo SFTP como las consultas HTTP a la API REST. Si bien `libssh2` ofrece un API más detallado para operaciones SSH/SFTP, `libcurl` unifica ambos protocolos bajo la misma interfaz (`curl_easy_setopt`), reduciendo la superficie de dependencias externas a una sola librería. Además, `libcurl` está disponible por defecto en Ubuntu 24.04 LTS como paquete `libcurl4-openssl-dev`, lo que simplifica la instalación en el entorno de evaluación.

En el apartado de Hash del código, el programa realiza un filtrado previo sobre las 15 millones de transacciones directamente en la memoria RAM, consolidando y des-duplicando la información a través de estructuras `std::unordered_set` locales por cada hilo. Este proceso de reducción de datos reduce el universo de trabajo de 15,000,000 de líneas a una colección de exactamente 2,921,115 clientes únicos, garantizando que no se hiciese ninguna consulta duplicada a internet y disminuyendo la carga sobre el servidor en más de un 80%.

Las tres fases paralelizadas (descarga, parseo y consultas API) utilizan `#pragma omp parallel for schedule(dynamic)`. La elección de `dynamic` frente a `static` se debe a que el trabajo por ítem no es el mismo para todos los hilos: los archivos CSV varían en tamaño, y los tiempos de respuesta de la API dependen de la red. Con reparto estático, los hilos que reciben archivos

grandes o respuestas lentas retrasan al grupo completo. Con reparto dinámico, cada hilo solicita un nuevo ítem en cuanto termina el anterior, equilibrando la carga de forma automática.

En la segunda fase, los hilos de OpenMP se dedican de forma exclusiva a la comunicación con la red mediante el uso de la biblioteca libcurl. El gran incremento en el rendimiento radica en la transición hacia peticiones por lotes (batching) y multiplexación asíncrona utilizando la interfaz curl_multi (CURLM). En lugar de procesar cliente por cliente, los identificadores únicos se agrupan en bloques definidos por configuración. Mediante el uso de variables persistentes por hilo (static thread_local ContextoHilo), cada hilo OpenMP administra su propio pool de descriptors de conexión reutilizables, manteniendo los canales abiertos bajo el protocolo HTTP Keep-Alive y eliminando el costo de handshake TCP y negociación SSL en cada iteración.

A la hora de conectarse con la API REST para conseguir el género, se requiere autenticación mediante JWT. El token se obtiene inicialmente y se reutiliza para las consultas posteriores (el token dura alrededor de 17 minutos y 30 segundos). Cuando la API responde con HTTP 401 debido a expiración del token, la función obtenerGeneroCliente() detecta esta condición y se dispara el proceso de renovación. Sin embargo, durante la ejecución paralela se observó que múltiples hilos pueden detectar simultáneamente la expiración y lanzar solicitudes de autenticación casi al mismo tiempo. Este comportamiento no afecta la correctitud del procesamiento ni genera errores en los resultados, ya que todos los hilos terminan obteniendo un token válido y continúan con la ejecución, pero sí introduce un costo temporal asociado a autenticaciones redundantes, aumentando algunos segundos el tiempo total del programa (5 a 10 segundos).

Para evitar sobrecargar el servidor, las descargas SFTP se reintentan hasta Config::MAX_RETRIES (5) veces con un retardo que se duplica en cada intento (2, 4, 8, 16... segundos). Esta política de backoff exponencial reduce la carga sobre el servidor remoto durante episodios de congestión o fallos transitorios, en contraste con reintentos inmediatos que podrían agravar la situación.

El parser valida cada fila contra las restricciones de tipo declaradas en la especificación (UUID v4, formato ISO 8601, enteros positivos, decimales). Se decidió implementar una corrección mínima para fechas con 16 caracteres (formato YYYY-MM-DDTHH:MM) añadiendo ":00" para normalizar a 19 caracteres, ya que por razones desconocidas, no utilizar 19 caracteres causa errores de exportación. Las filas que no superan la validación se descartan y se contabilizan, pero no se registran individualmente en el log para evitar archivos de bitácora excesivamente grandes.

El parser usa std::stringstream con getline en lugar de un split por índice o strtok. La lógica de separarLinea() usa getline(stream, valor, ';') sobre un stringstream, lo que maneja correctamente campos que terminan en ; (agrega un campo vacío al final) y strips de comillas envolventes. Es una elección deliberada frente a alternativas como strtok (no thread-safe,

destruye el string original) o `std::string::find` en un loop (más verboso y propenso a errores de índice).

IV. Diagrama de flujo o pseudocódigo del algoritmo paralelo

La ejecución sigue el orden:

1. Creación de carpeta de resultados (output/).
2. Conexión al servidor SFTP y listado de archivos con patrón reporte_*.csv.
3. Descarga paralela de archivos no presentes en disco (un hilo por archivo).
4. Descubrimiento local de archivos .csv en output/ y parseo paralelo.
5. Consulta paralela a la API REST con caché en memoria (tabla hash compartida).
6. Cálculo de promedios de monto por género y reporte de resultados.

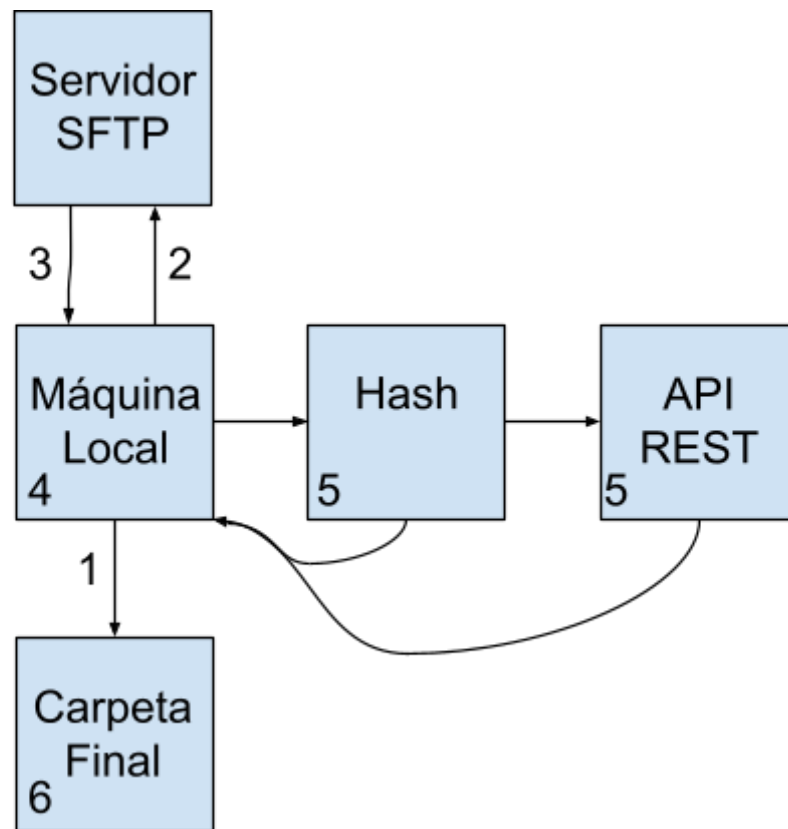


Imagen 1: Diagrama de flujo

ANÁLISIS DE RENDIMIENTO

I. Tiempo de ejecución

A la hora de probar el programa, ninguno de los integrantes tiene una máquina que lograse tener más de 12 hilos, debido a esto, el tiempo de ejecución con 32 hilos solo es una estimación.

Cabe recalcar que los anteriores tiempos de ejecución fueron calculados en distintas horas del día, con distintas velocidades de conexión (30 Mbps, 100 Mbps), lo cual causa variaciones en los tiempos de ejecución, debido a que la mayor cantidad del tiempo del programa se utiliza en la descarga.

Cantidad de Hilos	Tiempo de Ejecución (Horas)
1 (Secuencial)	~18+
4	7 a 12
12	3 a 5
32	1 a 2*

Tabla 2: Tiempo de ejecución respecto a la cantidad de hilos

Para mayor detalle de tiempo de ejecución se registró esta tabla que emplea el tiempo de las dos partes principales del programa en base a la cantidad de hilos mínimos y máximos que poseían los computadores de los integrantes de trabajo.

Tarea	4 hilos [seg]	12 hilos [seg]	Comparación
Descarga SFTP	2,915.81	1,430.58	≈ 51% <i>más rápido</i>
Parseo CSV	97.84	67.01	≈ 31% <i>más rápido</i>
Tiempo total	3,013.65	1,497.59	≈ 50% <i>de ahorro total</i>

Tabla 3: Detalle de tiempo de ejecución respecto a la cantidad de hilos

```

-> Caché poblada en 9608.66 segundos.

[3/3] Calculando promedios finales en memoria...

5. Cálculo de Métricas:
- Promedio de compras por género:
  - FEMENINO = 12073.9
  - MASCULINO = 12773.5

6. Salida:
- FEMENINO = 12073.9
- MASCULINO = 12773.5
- TIEMPO = 9622.74 segundos
[2026-06-15 03:35:34] [INFO] Métricas calculadas. FEM=12073.904663 MASC=12773.498840 TIEMPO=9622.742752s
=====
TIEMPO EXCLUSIVO DE API / MÉTRICAS: 9627.55 segundos.
=====

=== PROCESO FINALIZADO EXITOSAMENTE ===
Tiempo total global de ejecución: 11027.3 segundos

```

Imagen : Foto de pantalla de la terminal en la mejor ejecución obtenida el 14 - 06 - 2026.

Esta imagen muestra el mejor resultado obtenido en 12 hilos el día 14-06-2026 en la madrugada, con un tiempo de 9627.55 segundos (2 horas con 43 minutos) y unos 23 minutos en el tiempo de descarga, dando alrededor de 3 horas de ejecución. En la sección de dificultades encontradas y cómo se resolvieron se detalla la razón de porque no se logró replicar resultados adjuntando los archivos .txt en la entrega final en el repositorio.

RESULTADOS DE EJECUCIÓN

Con el dataset base entregado, se calcularon los siguientes promedios de gastos por género.

Género	Masculino	Femenino
Total Boleta	12.073,9	12.773,5

Tabla 4: Promedio de compras por género

DIFICULTADES ENCONTRADAS Y CÓMO SE RESOLVIERON

A lo largo del desarrollo del trabajo, hemos encontrado problemas a la hora de optimizar los tiempos de descarga. A pesar de realizar descarga paralela con múltiples hilos, los tiempos de descarga oscilan entre 30 y 60 minutos, con ciertos casos aislados teniendo tiempos de hasta 90 minutos. Cabe destacar que con 4 hilos, los tiempos de descarga se acercan a las 4 horas, pero con 12, los tiempos varían de 20 a 50 minutos.

A la hora de realizar parseo, surgió la idea de tener agentes de descarga, y agentes de parseo trabajando en paralelo al mismo tiempo con el objetivo de disminuir la secuencialidad de estos procesos, sin embargo, tras realizar una variedad de pruebas, los computadores utilizados siempre colapsaron, congelándose al quedarse sin RAM para continuar la tarea. Debido a esto, se decidió mantener la secuencialidad de ambos procesos.

El día de la entrega, se decidió realizar las últimas pruebas de funcionamiento del programa, sin embargo, la API REST dejó de funcionar alrededor de las 13:00 horas, esto causó un problema catastrófico. Cada vez, antes de iniciar el programa, se borra manualmente la carpeta final para evitar problemas de que el programa guarde información donde no debe, debido a esto, en medio de las pruebas, el programa fue incapaz de terminar su ejecución, no pudiendo generar un nuevo archivo de respuestas para la entrega del trabajo.

Además, cabe recalcar que días antes de la entrega la API REST estuvo muy saturada con consultas lo que provocó una modificación gigante respecto al tiempo total de ejecución código, fue prácticamente imposible hacer pequeñas modificaciones al final de la entrega y probarlas por esa razón.

CONCLUSIONES

En conclusión, la solución desarrollada permitió resolver el problema planteado mediante una aplicación en C++ capaz de descargar, procesar y analizar grandes volúmenes de datos de ventas desde archivos CSV, incorporando consultas a una API REST para complementar la información de cada cliente y calcular el promedio de compras según género. El diseño del sistema se estructuró de forma modular, separando responsabilidades como configuración, conexión SFTP, parseo de archivos, consulta a API, manejo de logs y procesamiento paralelo, lo que facilitó la organización del código, su mantenimiento y la identificación de errores durante la ejecución.

Respecto al diseño paralelo, el uso de OpenMP permitió optimizar las etapas más costosas del proceso, principalmente la descarga de archivos, el parseo de CSV y las consultas a la API. Además, la incorporación de una caché en memoria fue una decisión clave, ya que evitó consultas duplicadas para un mismo cliente y redujo significativamente la carga sobre el servidor externo. Esta estrategia permitió disminuir el universo de procesamiento desde aproximadamente 15 millones de transacciones a 2.921.115 clientes únicos, mejorando la eficiencia general de la solución.

Los resultados de rendimiento muestran que la implementación paralela logró una mejora importante frente a la versión secuencial. Mientras que el programa inicial podía tardar más de 18 horas, la versión paralela redujo el tiempo total a rangos aproximados de 7 a 12 horas con 4 hilos y de 3 a 5 horas con 12 hilos. En las mediciones más detalladas, el tiempo total bajó de 3.013,65 segundos con 4 hilos a 1.497,59 segundos con 12 hilos, lo que representa cerca de un 50% de ahorro total. Esto demuestra que el paralelismo aplicado fue efectivo, aunque el rendimiento final siguió dependiendo de factores externos como la velocidad de conexión, la disponibilidad del servidor SFTP y la estabilidad de la API.

En cuanto a los resultados funcionales, el programa logró calcular correctamente el promedio de compras por género a partir del dataset base, obteniendo un promedio de 12.073,9 para clientes masculinos y 12.773,5 para clientes femeninos. Esto confirma que la solución no solo mejoró el tiempo de procesamiento, sino que también cumplió con el objetivo principal del trabajo: procesar los datos, asociarlos al género correspondiente y entregar métricas claras para su análisis.

Finalmente, se concluye que la solución implementada cumple con los requerimientos principales del problema, ya que integra descarga remota, procesamiento de archivos CSV, consumo de API REST, uso de caché, manejo de errores y paralelismo con OpenMP. Sin embargo, también se identificaron limitaciones importantes, especialmente relacionadas con la dependencia de servicios externos y el consumo de recursos de memoria al intentar ejecutar varias fases simultáneamente. Por ello, una mejora futura podría considerar una arquitectura por lotes más controlada, mayor persistencia de caché, monitoreo de fallos de API y una mejor estrategia de recuperación ante interrupciones, con el fin de hacer el sistema aún más robusto y escalable.

ANEXOS

- (1) https://github.com/JoMi-MorV/problema_ComputacionParalela